

Table 1: Comparison among MemNN, ∂ ILP and the proposed NLM in family tree and graph reasoning, where m is the size of the testing family trees or graphs. Both ∂ ILP and NLM outperform the neural baseline and achieve perfect accuracy (100%) on test set. Note N/A mark means that ∂ ILP cannot scale up in 2-OutDegree.

Family Tree	MemNN		∂ ILP		NLM (Ours)	
	$m = 20$	$m = 100$	$m = 20$	$m = 100$	$m = 20$	$m = 100$
HasFather	99.9% / 99.9%	59.8% / 65.2%	100%	100%	100%	100%
HasSister	86.3% / 85.5%	59.8% / 66.4%	100%	100%	100%	100%
IsGrandparent	96.5% / 84.7%	97.7% / 63.7%	100%	100%	100%	100%
IsUncle	96.3% / 85.8%	96.0% / 64.0%	100%	100%	100%	100%
IsMGUncle	99.7% / 98.4%	98.4% / 81.7%	100%	100%	100%	100%
Graph	MemNN		∂ ILP		NLM (Ours)	
	$m = 10$	$m = 50$	$m = 10$	$m = 50$	$m = 10$	$m = 50$
AdjacentToRed	95.2% / 94.6%	93.1% / 91.9%	100%	100%	100%	100%
4-Connectivity	92.3% / 90.5%	81.3% / 88.0%	100%	100%	100%	100%
6-Connectivity	67.6% / 58.8%	43.9% / 67.9%	100%	100%	100%	100%
1-OutDegree	99.8% / 99.7%	78.6% / 81.2%	100%	100%	100%	100%
2-OutDegree	81.4% / 61.8%	96.7% / 87.7%	N/A	N/A	100%	100%

IsSon, IsDaughter, IsFather and IsMother. The goal of the task is to reason out other properties of family members or relations between them. Our results are summarized in Table 1.

For MemNN, we treat the problem of relation prediction as a question answering task. For example, to determine whether member x has a father in the family tree, we input $\text{id}(x)$ to MemNN as the question. MemNN then performs multiple queries to the memory and updates its hidden state. The finishing hidden state is used to classify whether $\text{HasFather}(x)$. For relations (binary predicates), the corresponding MemNN takes the concatenated embedding of $\text{id}(x)$ and $\text{id}(y)$ as the question.

For ∂ ILP, we take the grounded probability of the “target” predicate as the output; for an NLM with D layers, we take the corresponding group of output predicates at the last layer (for property prediction, we use tensor $O_D^{(1)}$ to represent unary predicates, while for relation prediction we use tensor $O_D^{(2)}$ to represent binary predicates) and classify the property or relation with a linear layer.

All models are trained on instances of size 20 and tested on instances of size 20 and 100 (size is defined as the number of family members). The models are trained with fully supervised learning (labels are available for all objects or pairs of objects). During the testing phase, the accuracy is evaluated (and averaged) on all objects (for properties such as HasFather) or pairs of objects (for relations such as IsUncle). MGUncle is defined as one’s maternal great uncle, which is also used by Differentiable Neural Computer (DNC) (Graves et al., 2016). We report the performance of MemNN in the format of Micro / Macro accuracy. We also try our best to replicate the setting used by Graves et al. (2016), and as a comparison, in the task of “finding” the MGUncle instead of “classifying”, DNC reaches the accuracy of 81.8%.

3.3 GENERAL GRAPH REASONING

We further extend the Family tree to general graphs and report the reasoning performance in Table 1.

We treat each node in the graph as an object (symbol). The (undirected) graph is fed into the model in the form of a “HasEdge” relation between nodes (which is an adjacent matrix). Besides, an extra property color represented by one-hot vectors is defined for every node. A node has the property of AdjacentToRed if it is adjacent to a red node by an outgoing edge. k -Connectivity is a relation between two nodes in the graph, which is true if two nodes are connected by a path with length at most k . A node has property k -OutDegree if its out-degree is exactly k . The N/A result of ∂ ILP in the 2-OutDegree task comes from its memory restriction (Evans & Grefenstette, 2018), where 3-ary intentional predicate is required. As an example, a human-written logic rule for 2-OutDegree can be $\neg\text{OutDegree}(a) \leftarrow \exists b \exists c \forall d \text{HasEdge}(a, b) \wedge \text{HasEdge}(a, c) \wedge \neg\text{HasEdge}(a, d)$ where a, b, c and d are distinct nodes in the graph.

All models are trained on instances of size 10 and tested on instances of size 10 and 50 (size is defined as the number of nodes in the graph).

Table 2: Comparison between MemNN and the proposed NLM in the blocks world, sorting integers, and finding shortest paths, where m is the number of blocks in the blocks world environment or the size of the arrays/graphs in sorting/path environment. Both models are trained on instance size $m \leq 12$ and tested on $m = 10$ or 50 . The performance is evaluated by two metrics and separated by “/”: the probability of completing the task during the test, and the average Moves used by the agents when they complete the task. There is no result for ∂ ILP since it fails to scale up. MemNN fails to complete the blocks world within the maximum $m \times 4$ Moves.

Task	MemNN		NLM (Ours)	
	$m = 10$	$m = 50$	$m = 10$	$m = 50$
BlocksWorld	0% / N/A	0% / N/A	100% / 12	100% / 84
Sorting	100% / 22	90% / 986.6	100% / 8	100% / 45
Path	45% / 13.3	12% / 42.7	100% / 4	100% / 4

3.4 BLOCKS WORLD

We also test NLM’s capability of decision making in the classic blocks world domain (Nilsson, 1982; Gupta & Nau, 1992) by slightly extending the model to fit the formulation of Markov Decision Process (MDP) in reinforcement learning.

Shown in Figure 1, an instance of the blocks world environment contains two worlds: the initial world and the target world, each containing the ground and m blocks. The task is to take actions in the operating world and make its configuration the same as the target world. The agent receives positive rewards only when it accomplishes the task and the sparse reward setting brings significant hardness. Each object (blocks or ground) can be represented by four properties: `world_id`, `object_id`, `coordinate_x`, `coordinate_y`. The ground has a fixed coordinate $(0, 0)$. The input is the result of the numeral comparison among all pairs of objects (may come from different worlds). For example, in x -coordinate, the comparison produces three relations for each object pair (i, j) , $i \neq j$: `Left`(i, j) (whether i is to the left of j , or $\mathbb{1}[x_i < x_j]$), `SameX`(i, j) and `Right`(i, j).

The only operation is `Move`(i, j), which moves object i onto the object j in the operating world if i is movable and j is placeable. If the operation is invalid, it will have no effect; otherwise, the action takes effect and the state represented as coordinates will change accordingly. In our setting, an object i is movable iff it is not the ground and there are no blocks on it, i.e. $\forall_j \neg(\text{Up}(i, j) \wedge \text{SameX}(i, j))$. Object i is placeable iff it is the ground or there are no blocks on it.

To avoid the ambiguity of the x -coordinates while putting blocks onto the ground, we set the x coordinate of block i to be i when it is placed onto the ground. The action space is $(m + 1) \times m$ where m is the number of blocks in the world and $+1$ comes from the “ground”. For both MemNN and NLM, we apply a shared MLP on the output relational predicates of each pair of objects $O_D^{(2)}(x, y)$ and compute an action score $s(x, y)$. The probability for `Move`(x, y) is $\propto \exp s(x, y)$ (by taking a Softmax). The results are summarized in Table 2. For more discussion on the confidence bounds of the experiments, please refer to Appendix B.6.

3.5 GENERAL ALGORITHMS

We further show NLM’s ability to excel at algorithmic tasks, such as `Sorting` and `Path`. We view an algorithm as a sequence of primitive actions and cast as a reinforcement learning problem.

Sorting. We first consider the problem of sorting integers. Given a length- m array a of integers, the algorithm needs to iterative swap elements to sort the array in ascending order. We treat each slot in the array as an object, and input their index relations (whether $i < j$) and numeral relations (whether $a[i] < a[j]$) to NLM or MemNN. The action space is $m \times (m - 1)$ indicating the pair of integers to be swapped. Table 2 summarizes the learning performance.

As the comparisons between all pairs of elements in the array are given to the agent, sorting the array within the maximum number of swaps is an easy task. A trivial solution is to randomly swap an inversion⁴ in the array at each step.

Beyond being able to generalize to arrays of arbitrary length, with different hyper-parameters and random seeds, the learned algorithms can be interpreted as Selection-Sort, Bubble-Sort, *etc.* We include videos demonstrating some learned algorithms in our website.⁵

⁴[https://en.wikipedia.org/wiki/Inversion_\(discrete_mathematics\)](https://en.wikipedia.org/wiki/Inversion_(discrete_mathematics))

⁵<https://sites.google.com/view/neural-logic-machines>

Path finding. We also test the performance of finding a path (single-source single-target path) in a given graph as a sequential decision-making problem in reinforcement learning environment. Given an undirected graph represented by its adjacency matrix as relations, the algorithm needs to find a path from a start node s (with property $\text{IsStart}(s) = \text{True}$) to the target node t (with property $\text{IsTarget}(t) = \text{True}$). To restrict the number of deduction steps, we set the maximum distance between s and t to be 5 during the training and set the distance between s and t to be 4 during the testing, which replicates the setting of Graves et al. (2016). Table 2 summarizes the result.

Path task here can be seen as an extension of bAbI task 19 (path finding) (Weston et al., 2015) with symbolic representation. As a comparison with graph neural networks, Li et al. (2016) achieved 99% accuracy on the bAbI task 19. Contrastively, we formulate the shortest path task as a more challenging reinforcement learning (decision-making) task rather than a supervised learning (prediction) task as in Graves et al. (2016). Specifically, the agent iteratively chooses the next node *next* along the path. At the next step, the starting node will become *next* (at each step, the agent will move to *next*). As a comparison, in Graves et al. (2016), Differentiable Neural Computer (DNC) finds the shortest path with probability 55.3% in a similar setting.

4 RELATED WORKS AND DISCUSSIONS

ILP and relational reasoning. Inductive logic programming (ILP) (Muggleton, 1991; 1996; Friedman et al., 1999) is a paradigm for learning logic rules derived from a limited set of rule templates from examples. Being a powerful way of reasoning over discrete symbols, it is successfully applied to various language-related problems, and has been integrated into modern learning frameworks (Kersting et al., 2000; Richardson & Domingos, 2006; Kimmig et al., 2012). Recently, Evans & Grefenstette (2018) introduces a differentiable implementation of ILP which works with connectionist models such as CNNs. Sharing a similar spirit, Rocktäschel & Riedel (2017) introduces an end-to-end differentiable logic proving system for knowledge base (KB) reasoning. A major challenge of these approaches is to scale up to a large number of complex rules. Searching a rule as complex as our *ShouldMove* example in Appendix E from scratch is beyond the scope of most systems that use weighted symbolic rules generated from templates.

As shown in Section 2.4, both computational complexity and parameter size of the NLM grow *polynomially* w.r.t. the number of allowed predicates (in contrast to the *exponential* dependence in ∂ILP (Evans & Grefenstette, 2018)), but factorially w.r.t. the breadth (max arity, same as ∂ILP). Therefore, our method can deal with more complex tasks such as the blocks world which requires using a large number of intermediate predicates, while ∂ILP fails to search in such a large space.

Our paper also differs from existing approaches on using neural networks to augment symbolic rule induction (Lippi & Frasconi, 2009; Manhaeve et al., 2018). Specifically, we have *no* rule designed by humans as the input or the knowledge base for the model. NLMs are general neural architectures for learning lifted rules from only input-output pairs.

Our work is also related to symbolic relational reasoning, which has a wide application in processing discrete data structures such as knowledge graphs and social graphs (Zhu et al., 2014; Kipf & Welling, 2017; Zeng et al., 2017; Yang et al., 2017). Most symbolic relational reasoning approaches (e.g., Yang et al., 2017; Rocktäschel & Riedel, 2017) are developed for KB reasoning, in which the predicates on both sides of a rule is known in the KB. Otherwise, the complexity grows exponentially in the number of used rules for a conclusion, which is the case in the blocks world. Moreover, Yang et al. (2017) considers rules of the form $\text{query}(Y, X) \leftarrow R_n(Y, Z_n) \wedge \dots \wedge R_1(Z_1, X)$, which is not for general reasoning. The key of Rocktäschel & Riedel (2017) and Campero et al. (2018) is to learn subsymbolic embeddings of entities and predicates for efficient KB completion, which differs from our focus. While NLMs can scale up to complex rules, the number of objects/entities or relations should be bounded as a small value (e.g., < 1000), since all predicates are represented as tensors. This is, to some extent, in contrast with the systems developed for knowledge base reasoning. We leave the scalability of NLMs to large entity sets as future works.

Besides, modular networks (Andreas et al., 2016; 2017; Mascharka et al., 2018) are proposed for the reasoning over subsymbolic data such as images and natural language question answering. Santoro et al. (2017) implements a visual reasoning system based on “virtual” objects brought by receptive